TaxHub — Architecture

Tax-form finding and tax-law traceability for a fund-management back office. TaxHub's primary job is to help the back office find the **correct tax form** to file across the jurisdictions JTC Group operates in; secondarily it scrapes official tax documents, captures **every version**, detects and explains **every change**, traces guidance back to the **primary law** it interprets, and answers questions over the resulting citation/change graph.

This document describes the implemented system: components, data model, data flow, the agent orchestrator + graph-RAG retrieval, configuration, and the **target production deployment on Microsoft Azure** — Azure AI Foundry (LLMs + Agents), Azure Blob Storage (document/PDF store), and a managed **Neo4j AuraDB** graph. For a quick start see the top-level `README.md`.

1. Current features

Area	Feature	Where
Forms	Config-driven form catalogue, 26 jurisdictions	`config/tax_forms.yaml`, `ingest/forms.py`
	Form metadata + PDF download / discover-all over authority indexes	`ingest/forms.py`, `ingest/scrapers/`
		`ingest/forms.py:forms_tree`

Area	Feature	Where
	Forms Tree taxonomy (jurisdiction → category → type → form)	
Ingest	Config-driven document catalogue	`config/tax_sources.yaml`, `ingest/cli.py`
	HTTP + headless-browser fetch (JS/UA-gated sites)	`ingest/fetch.py`, `ingest/scrapers/`
	HTML + PDF text extraction, normalised + SHA-256 hashed	`ingest/fetch.py`
Versioning	Immutable version appended only when the content hash changes	`storage/*`, `ingest/cli.py`
	Consecutive-version diffs (added/removed chars + diff text)	`ingest/cli.py`, `rag/llm.py`
AI	Grok plain-English change summary + back-office impact	`rag/llm.py`
	Citation extraction (guidance → statutory instruments)	`rag/llm.py`
	Graceful degradation when no `XAI_API_KEY` is set	`rag/llm.py`, `rag/retrieval.py`, `agents/orchestrator.py`
Agents	LangGraph tool-calling orchestrator routing to 4 specialist agents	`agents/orchestrator.py`, `agents/tools.py`
	SSE token + tool-step streaming to the UI	`agents/sse.py`, `agents/orchestrator.py`
Storage	Backend-neutral `Storage` interface	`storage/base.py`
	Neo4j graph backend (default)	`storage/neo4j_store.py`
	SQLite / Postgres relational backend	`storage/sqlite_store.py`
	One-line backend switch via `DATA_STORAGE`	`storage/__init__.py`
	SQLite → Neo4j backfill migration	`scripts/migrate_sqlite_to_neo4j.py`
Retrieval	Full-text seed → graph expansion → Grok answer with cited sources	`rag/retrieval.py`
	Vector + hybrid (RRF) retrievers over fastembed chunk embeddings	`rag/retrieval.py`, `rag/embeddings.py`
	Chunk-embedding backfill	`scripts/embed_backfill.py`
Web app		`web/app.py`

Area	Feature	Where
	FastHTML 3-pane agentic app: Forms Tree, AI chat, changes/PDF viewer	
CLI	`--list / --jurisdiction / --all / --changes / --stats / --no-ai`	`ingest/cli.py`
Ops	Healthcheck, Dockerfile, Coolify-labelled compose, user-owned local Neo4j	`docker-compose.yaml` , `scripts/neo4j_local.sh`
Tests	Cross-backend storage contract + no-network RAG smoke	`tests/`
Evals	Retriever comparison harness over a ground-truth set	`evals/`

Web routes (`web/app.py` , port 5011)

Route	Purpose
`/health`	Liveness probe (used by the compose healthcheck)
`/login` , `/logout`	Session auth (admin seeded by `init_db`)
`/`	3-pane app: Forms Tree + AI Assistant chat + changes newsfeed
`/chat` (POST)	SSE stream from the LangGraph orchestrator (or shortcut routing)
`/dashboard`	Corpus stats + jurisdictions with document counts
`/jurisdictions` , `/jurisdiction/{code}`	Jurisdiction list / documents in a jurisdiction
`/documents` , `/upload`	Stored-PDF browser with filters; upload a PDF to the volume
`/document/{doc_id}`	Version history, changes, citations for a document
`/form/{form_id}` , `/form-pdf/{form_id}`	Form metadata + embedded PDF viewer
`/changes` , `/api/feed`	Recent changes across all jurisdictions / feed partial
`/help` , `/user-guide-pdf`	In-app help + the generated User Guide PDF

Jurisdiction coverage

The form catalogue (`config/tax_forms.yaml`) covers **26 jurisdictions** — the six live-MVP fund domiciles plus an expansion to all JTC Group office jurisdictions:

> **Live MVP:** Jersey (JE), Guernsey (GG), Luxembourg (LU), Ireland (IE), Cayman Islands (KY), British Virgin Islands (VG).
>
> **Expansion:** Isle of Man (IM), Mauritius (MU), Malta (MT), Cyprus (CY), Netherlands (NL), Switzerland (CH), United Kingdom (GB), Germany (DE), Austria (AT), Poland (PL), United States (US), Hong Kong (HK), Singapore (SG), Malaysia (MY), New Zealand (NZ), United Arab Emirates (AE), Bahamas (BS), Brazil (BR), South Africa (ZA), Bermuda (BM).

Each form is labelled by **filing type** — `downloadable` (a PDF to complete), `online` (web portal / e-file), or `reference` (guidance, not a fileable form). Most jurisdictions file online, but a subset publishes **downloadable PDFs** — notably the US (IRS), Hong Kong, New Zealand, Poland, Austria, Bermuda, Luxembourg, and Guernsey — which TaxHub fetches and stores locally for the in-app PDF viewer.

> The document/legislation catalogue (`config/tax_sources.yaml`) used for versioning + graph-RAG currently tracks the original six MVP jurisdictions.

---

2. Architecture — Azure target deployment

The production target runs the containerised app on **Azure**, with the LLM/agent layer on **Azure AI Foundry**, documents in **Azure Blob Storage**, and the graph on a **managed Neo4j AuraDB** instance.

```
flowchart TB
    USER(["Back-office user"])
    SRC[("Official tax-authority sources<br/>26 JTC Group jurisdictions<br/>(forms, legislation, guidance)")]

    subgraph Azure["Microsoft Azure"]
        subgraph ACA["Azure Container Apps"]
            APP["TaxHub container<br/>FastHTML 3-pane app<br/>web/ · agents/ · rag/ · ingest/"]
        end
        subgraph FOUNDRY["Azure AI Foundry"]
            MODELS[["Foundry models<br/>chat + reasoning (OpenAI-compatible)"]]
            FAGENTS[["Foundry Agents<br/>orchestrator + specialist tools"]]
        end
        BLOB[("Azure Blob Storage<br/>form PDFs + raw documents")]
        KV["Azure Key Vault<br/>secrets / connection strings"]
    end

    AURA[("Neo4j AuraDB<br/>managed graph database<br/>neo4j+s://…")]

    USER -->|HTTPS| APP
    APP -->|route + stream| FAGENTS
    FAGENTS --> MODELS
    APP -->|graph read/write · Bolt+TLS| AURA
```

```
    APP -->|store / serve PDFs| BLOB
    APP -->|scheduled scrape| SRC
    SRC -->|PDFs + HTML| APP
    APP -.->|reads secrets| KV
```

The application is backend- and provider-neutral, which is what makes this target clean to hit:

- **LLM/agents** — the orchestrator (`agents/`) and RAG layer (`rag/llm.py`) talk to an **OpenAI-compatible** endpoint, so the same code points at Azure AI Foundry model deployments (and Foundry Agents) in production by setting the Azure endpoint + key; no code change versus the local LLM.
- **Documents** — form PDFs and raw captures live in **Azure Blob Storage** in production (`DOC_STORAGE=blob`); the local filesystem volume is the dev fallback.
- **Graph** — `DATA_STORAGE=neo4j` points at **managed Neo4j AuraDB** over Bolt+TLS; the same `Storage` interface backs both backends, so no caller (app, agents, retrieval, CLI) knows which database is live.

`taxstore.py` is a thin compat shim re-exporting the active store from `storage.get_store()`. For the internal module/data-flow view (ingestion → storage → serving), see §3 and §4.

---

3. Scrape & versioning flow

How one document moves from source to a stored, diffed, AI-summarised version:

```
sequenceDiagram
    participant CLI as ingest/cli.py
    participant F as ingest/fetch.py
    participant S as Storage backend
    participant AI as rag/llm.py (Grok)

    CLI->>S: start_run(jurisdiction)
    CLI->>F: fetch(url, mode=http|browser)
    F->>F: extract text (html|pdf) + normalise
    F->>F: sha256(normalised text)
    F-->>CLI: text, content_hash
    CLI->>S: latest_version(doc_id)
    alt hash unchanged
        CLI->>S: mark_checked(doc_id)
    else hash changed (or first capture)
        CLI->>S: add_version(doc_id, hash, text)
        CLI->>CLI: diff(prev_text, new_text)
        CLI->>AI: summarise change + impact
        AI-->>CLI: ai_summary, ai_impact
        CLI->>S: record_change(from_v, to_v, diff, ai_*)
        CLI->>AI: extract citations(text)
        AI-->>CLI: instruments + locators
        CLI->>S: add_citations(doc_id, version, citations)
```

```
    end
    CLI->>S: finish_run(checked, new, changed, errors)
```

Versions are **immutable and append-only** — the audit trail of how the law changed. A change row is written only on an actual content-hash difference.

4. Graph data model (Neo4j)

```
graph LR
    J["(:Jurisdiction)<br/>code, name, region, authority"]
    D["(:Document)<br/>uid, doc_key, title, reference, url"]
    V["(:Version)<br/>uid, version_no, content_hash, text"]
    C["(:Change)<br/>uid, change_type, diff, ai_summary, ai_impact"]
    I["(:Instrument)<br/>key, name"]
    CH["(:Chunk)<br/>ord, chunk_text, embedding"]
    F["(:Form)<br/>uid, form_key, title, filing_type, file_path"]

    J -- HAS_DOCUMENT --> D
    J -- HAS_FORM --> F
    D -- HAS_VERSION --> V
    D -- CURRENT_VERSION --> V
    D -- HAS_CHANGE --> C
    C -- FROM_VERSION --> V
    C -- TO_VERSION --> V
    V -- "CITES {locator, snippet}" --> I
    V -- HAS_CHUNK --> CH
```

Design notes:

- **Stable integer ids** (`uid`) are minted from a `(:Counter)` node, so `/document/{id}` and `/form/{id}` URLs work identically across backends and the deprecated `id()` function is avoided — portable straight to AuraDB.
- **Citations are first-class edges** (`(:Version)-[:CITES]->(:Instrument)`), which is exactly what graph-RAG traverses.
- **Forms are first-class nodes** (`(:Jurisdiction)-[:HAS_FORM]->(:Form)`), classified by category + form_type to build the Forms Tree, and distinct from legislation/guidance `Document` nodes.
- **Chunk vectors** (`(:Version)-[:HAS_CHUNK]->(:Chunk)`) carry per-passage embeddings for vector/hybrid retrieval, backed by a Neo4j native `VECTOR INDEX` (cosine).
- **Null properties are re-materialised** on read (Neo4j omits null keys; the app expects every column present) so the two backends are behaviourally identical.
- **Version-tolerant DDL**: `init_db` tries 5.x syntax (`REQUIRE`, `CREATE FULLTEXT INDEX`) and falls back to 4.x (`ASSERT`, `db.index.fulltext` procedure), so the same code runs on local Neo4j 4.x and AuraDB 5.x.

The relational backend stores the same shape as tables: `jurisdictions`, `tax_documents`, `document_versions`, `document_changes`, `citations`, `version_chunks`, `tax_forms`, `scrape_runs`, `users`, plus chat history (`chat_sessions`, `chat_messages`).

---

5. Agents + Graph-RAG — the assistant

The web app's chat routes every message to a **LangGraph tool-calling orchestrator** (`agents/orchestrator.py`, `create_react_agent`). The orchestrator picks the right specialist agent — exposed as the four tools in `agents/tools.py` — and streams tokens and tool steps back over SSE:

Tool	Job
`document_agent`	Find the correct tax FORM(s) for a need (the primary use case)
`law_agent`	Tax-LAW Q&A via graph-RAG over the tracked corpus, with citations
`metadata_agent`	Structured lookup: forms / deadlines for a jurisdiction or category
`changes_agent`	Recent changes to tracked documents

`law_agent` delegates to graph-RAG retrieval:

```
flowchart LR
    Q(["Question"]) --> SEED
    subgraph Retrieve["rag.retrieval — fulltext / vector / hybrid"]
        SEED["1. Seed<br/>full-text (Neo4j Lucene / SQLite scan)<br/>and/or vector
(fastembed chunks)<br/>fused by reciprocal-rank (hybrid)"]
        EXP["2. Graph expansion<br/>cited instruments + latest change"]
        SEED --> EXP
    end
    EXP --> CTX["Numbered, grounded context blocks"]
    CTX --> GEN["rag.retrieval.answer()<br/>Grok generation"]
    GEN --> A(["Answer + cited sources"])
    GEN -. "no XAI_API_KEY" .-> DEG["Degrade: return sources only"]
```

Retrieval and generation are **deliberately separated** (`Retriever` ABC + `answer()`). Three retrievers implement the ABC — `GraphFullTextRetriever`, `VectorRetriever`, and `HybridRetriever` — selected by `RAG_RETRIEVER` (default `hybrid`). **Embeddings are local fastembed** (`BAAI/bge-small-en-v1.5`, 384-dim; `rag/embeddings.py`), chunked per passage and stored as `(:Chunk)` nodes / `version_chunks` rows. Vector and hybrid retrievers **degrade to full-text** when no embeddings/chunks are present, so the assistant works with or without a backfill (`scripts/embed_backfill.py`).

---

6. Environment variables

Copy `.env.example` → `.env` (gitignored). All variables:

Variable	Required	Default	Purpose
`DATA_STORAGE`	yes	`neo4j`	Active backend: `neo4j` or `sqlite`
`DB_URL`	sqlite mode	`sqlite:///taxhub.db`	SQLite/Postgres DSN; also the migration **source**
`NEO4J_URI`	neo4j mode	`bolt://localhost:7687`	Bolt URI. AuraDB: `neo4j+s://<id>.databases.neo4j.io`
`NEO4J_USER`	neo4j mode	`neo4j`	Neo4j username
`NEO4J_PASSWORD`	neo4j mode	—	Neo4j password
`NEO4J_DATABASE`	no	`neo4j`	Target database name
`LLM_PROVIDER`	no	`xai`	Provider selector: `xai` (dev) or `azure` (production / Azure AI Foundry)
`LLM_API_KEY` / `XAI_API_KEY`	no	—	LLM key; absent → AI features degrade gracefully
`LLM_BASE_URL` / `XAI_BASE_URL`	no	`https://api.x.ai/v1`	OpenAI-compatible base URL
`GROK_MODEL`	no	`grok-4-1-fast-reasoning`	Model / deployment id

Azure target (production) — set these for the Azure AI Foundry + Blob deployment:

Variable	Required	Purpose
`AZURE_AI_FOUNDRY_ENDPOINT`	Azure	Foundry project / model endpoint (OpenAI-compatible)
`AZURE_AI_FOUNDRY_API_KEY`	Azure	Foundry key (or use Managed Identity)
`FOUNDRY_MODEL`	Azure	Deployed model name (chat)
`FOUNDRY_AGENT_ID`	optional	Foundry Agent id, if routing via Foundry Agents
`DOC_STORAGE`	Azure	`blob` in production (`local` for dev)
`AZURE_STORAGE_CONNECTION_STRING`	Azure	Blob Storage connection string (or Managed Identity)
`BLOB_CONTAINER`	Azure	Container for form PDFs + raw docs (e.g. `taxhub-docs`)
`RAG_RETRIEVER`	no	`hybrid`
`EMBEDDINGS`	no	`fastembed`

Variable	Required	Purpose
`EMBED_MODEL`	no	`BAAI/bge-small-en-v1.5`
`EMBED_DIM`	no	`384`
`ADMIN_EMAIL`	yes	`admin@example.com`
`ADMIN_PASSWORD`	yes	*(set in env)*
`APP_SECRET`	yes (prod)	*(set in env)*
`TAXHUB_PUBLIC`	no	`0`
`PORT`	no	`5011`

7. Deployment — Microsoft Azure

The production target is Azure end-to-end. The architecture diagram in §2 shows the runtime; this section is the how-to.

7.1 Azure service mapping

Concern	Azure service	Notes
App runtime	**Azure Container Apps**	Runs the existing `Dockerfile`; scales to zero; built-in HTTPS ingress + `/health` probe. (Azure App Service for Containers is an equally valid host.)
LLM + agents	**Azure AI Foundry**	Model deployment (chat/reasoning) consumed over the OpenAI-compatible endpoint; optionally Foundry **Agents** for orchestration.
Document store	**Azure Blob Storage**	Form PDFs + raw captures (`DOC_STORAGE=blob`).
Graph database	**Neo4j AuraDB** (managed)	Reached over `neo4j+s://` (Bolt+TLS). Azure Marketplace offers managed Aura, or use Neo4j Aura directly.
Secrets	**Azure Key Vault**	Foundry key, AuraDB password, `APP_SECRET`, Blob connection string — referenced as Container App secrets.
Registry	**Azure Container Registry**	Stores the built image pulled by Container Apps.
Scheduled scrape	**Container Apps Job** (cron)	Runs `ingest/cli.py --all` on a schedule to accrue change history.

7.2 Provision & deploy

```
# 0. Variables
RG=taxhub-rg; LOC=westeurope; ACR=taxhubacr; APP=taxhub

# 1. Resource group + container registry, then build & push the image
az group create -n $RG -l $LOC
az acr create -n $ACR -g $RG --sku Basic --admin-enabled true
az acr build -r $ACR -t taxhub:latest .            # builds the repo Dockerfile

# 2. Blob Storage for documents
az storage account create -n taxhubdocs -g $RG -l $LOC --sku Standard_LRS
az storage container create --account-name taxhubdocs -n taxhub-docs

# 3. Azure AI Foundry: create a project + deploy a chat model in the
#    Foundry portal (ai.azure.com); copy the endpoint + key (and Agent id if used).

# 4. Container Apps environment + app
az containerapp env create -n taxhub-env -g $RG -l $LOC
az containerapp create -n $APP -g $RG --environment taxhub-env \
  --image $ACR.azurecr.io/taxhub:latest --target-port 5011 --ingress external \
  --secrets aura-pass=<AURADB_PASSWORD> foundry-key=<FOUNDRY_KEY> \
            blob-cs=<BLOB_CONNECTION_STRING> app-secret=<RANDOM> \
  --env-vars DATA_STORAGE=neo4j LLM_PROVIDER=azure DOC_STORAGE=blob \
             NEO4J_URI=neo4j+s://<id>.databases.neo4j.io \
             NEO4J_USER=<id> NEO4J_DATABASE=<id> \
             NEO4J_PASSWORD=secretref:aura-pass \
             AZURE_AI_FOUNDRY_ENDPOINT=<endpoint> FOUNDRY_MODEL=<deployment> \
             AZURE_AI_FOUNDRY_API_KEY=secretref:foundry-key \
             AZURE_STORAGE_CONNECTION_STRING=secretref:blob-cs \
             BLOB_CONTAINER=taxhub-docs APP_SECRET=secretref:app-secret \
             ADMIN_EMAIL=<email> ADMIN_PASSWORD=secretref:app-secret
```

7.3 Managed Neo4j AuraDB

1. Create an AuraDB instance (Azure Marketplace "Neo4j Aura" or https://console.neo4j.io); **download the credentials file — shown once**.
2. Wire the four `NEO4J_*` values into the Container App (step 4 above): `NEO4J_URI=neo4j+s://<id>.databases.neo4j.io NEO4J_USER=<id> # see gotcha NEO4J_PASSWORD=<generated> NEO4J_DATABASE=<id> # see gotcha` `neo4j+s://` is Bolt over TLS (required by Aura). **Gotcha:** on AuraDB *Free*, the username **and** database name are the **instance id** (e.g. `05a16101`), not `neo4j` — connecting to database `neo4j` fails with `DatabaseNotFound`. On Professional they are typically `neo4j`. Trust the downloaded credentials file either way.
3. Initialise the schema (idempotent, version-tolerant 4.x/5.x DDL) — it runs on first app start (`init_db`), or on demand: `bash az containerapp exec -n $APP -g $RG --command "python3.12 taxstore.py"`
4. Backfill from an existing SQLite corpus, if any: `bash DB_URL=sqlite:///taxhub.db python3.12 scripts/migrate_sqlite_to_neo4j.py --wipe`

5. Schedule the scraper as a Container Apps **Job** (cron) running `ingest/cli.py --all` so change history accrues over time.

No application code changes are needed to hit this target — only environment variables: `LLM_PROVIDER=azure` (Foundry), `DOC_STORAGE=blob` (Blob Storage), and `DATA_STORAGE=neo4j` with the four `NEO4J_*` values (AuraDB).

---

8. Testing

```
python3.12 -m pytest tests/ -q
```

- `tests/test_storage.py` — one contract suite parametrised over **both** backends, so `DATA_STORAGE` can't silently change behaviour. The Neo4j case skips if no Neo4j is reachable and cleans up its own `T9` subgraph.
- `tests/test_rag.py` — no-network RAG smoke (seeds a temp SQLite store, stubs Grok off, asserts retrieval + graceful degradation).

---

9. Roadmap — what to do next

Ordered by leverage:

1. **Deploy to Azure** (§7) — Container Apps + Azure AI Foundry + Blob Storage + managed AuraDB. First real production target.
2. **Schedule the scraper** (daily/weekly cron) so change history accrues — the product's core value is the *change* trail, which only exists once scrapes run repeatedly.
3. **Backfill embeddings at scale** — `VectorRetriever` / `HybridRetriever` and `scripts/embed_backfill.py` already exist (AuraDB native vector index, local fastembed). Run the backfill across the full corpus so `/chat` law answers are semantic-by-default rather than degrading to full-text.
4. **Change digest / alerts** — email or in-app digest of recent changes per jurisdiction (the "back-office alert" use case).
5. **Harden gated sources** — JS/UA-gated authority sites (e.g. Jersey, Guernsey) still under-fetch some forms; revisit the browser fetch strategy.
6. **Broaden the document/legislation catalogue** — `config/tax_sources.yaml` tracks only the six MVP jurisdictions for versioning + graph-RAG; extend it toward the 26 jurisdictions already in the form catalogue.
7. **Auth & multi-user** — move beyond the single seeded admin if more than the back-office team needs access.